

DeliverEats

Team Collaboration & Standards Guide

Kareem • Sarhan • Hanaa • Basmala

This document is the single source of truth for how the team writes code, names things, uses Git, and communicates. Everyone must read and follow it. When in doubt, refer here first.

1. Team ownership map

Each person owns specific modules. If you need to touch someone else's module, talk to them first.

Member	Modules owned	Key responsibilities
Kareem	Auth, Orders, Payments, Surge Pricing	DB schema, state machine, Stripe Connect, surge engine
Sarhan	Dispatch, GPS, Real-time, Queues	Rider assignment, Redis, Pusher, WebSockets, security
Hanaa	Restaurant, Menu, Reviews, Dashboard	Restaurant CRUD, menu management, analytics, ratings
Basmala	Rider Portal, Notifications, Tests, Docs	Rider APIs, notification jobs, feature tests, user guides

Rule: Never modify another person's core module without creating a PR and tagging them as reviewer. Cross-module work = always coordinate first.

2. Naming conventions

These rules apply to every file, variable, class, column, and route in the project. No exceptions.

2.1 Database columns — always snake_case

// CORRECT

```
restaurant_id    is_open    delivery_fee    surge_multiplier
created_at       updated_at  rider_id       payment_status

// WRONG — never do these
restaurantId     isOpen      deliveryFee     surgeMultiplier
RestaurantId     IsOpen
```

2.2 PHP variables and methods — always camelCase

```
// CORRECT
$orderId         $riderId         $surgeMultiplier
calculateSurge() assignRider() processPayment()

// WRONG
$order_id        $rider_id        calculate_surge()
```

2.3 PHP classes — always PascalCase

```
// CORRECT
OrderService     SurgeStrategy     RiderDispatchJob
FlatSurgeStrategy MultiplierStrategy OrderStateMachine

// WRONG
orderService     surge_strategy     rider_dispatch_job
```

2.4 API routes — always kebab-case, versioned under /api/v1/

```
// CORRECT
GET    /api/v1/menu-items
POST   /api/v1/orders
PUT    /api/v1/rider-location
GET    /api/v1/restaurant-dashboard
POST   /api/v1/order-reviews

// WRONG
GET    /api/menuItems
POST   /api/Orders
GET    /api/v1/restaurantDashboard
```

2.5 JSON API responses — always snake_case keys

```
// CORRECT — what every API response must look like
{
  "order_id": 42,
  "restaurant_id": 7,
  "delivery_fee": 15.00,
```

```

    "surge_fee": 5.00,
    "status": "placed",
    "created_at": "2024-01-01T12:00:00Z"
  }

  // WRONG
  { "orderId": 42, "restaurantId": 7, "deliveryFee": 15 }

```

2.6 Laravel files — standard naming

Type	Convention + example
Models	PascalCase singular — Order, MenuItem, RiderLocation
Controllers	PascalCase + Controller — OrderController, RiderController
Services	PascalCase + Service — OrderService, DispatchService
Jobs (queues)	PascalCase + Job — AssignRiderJob, RecalculateSurgeJob
Events	PascalCase past tense — OrderPlaced, RiderAssigned
Migrations	snake_case timestamp — 2024_01_01_000000_create_orders_table
Enums / constants	UPPER_SNAKE_CASE — ORDER_PLACED, RIDER_AVAILABLE

3. API contracts — agree before coding (Ignoreeeeeeeeeee)

Any endpoint that crosses between two people's modules must be written down and agreed before either person codes it. Use this format:

```

METHOD  /api/v1/route
Owner:  [name of person who builds this endpoint]
Called by: [name of person who calls it]

Request body:
  { field: type, field: type }

Success response (200):
  { field: type, field: type }

Error responses:
  422 — validation failed
  403 — unauthorized role

```

Critical cross-module contracts

These are the endpoints where two people's work must connect. Both people must agree on these before coding:

Endpoint	From → To	Key fields to agree on
POST /api/v1/orders	Kareem → Sarhan	order_id, restaurant_id, delivery_address — Sarhan needs these to dispatch a rider
PUT /api/v1/rider-location	Sarhan → Kareem	rider_id, latitude, longitude, order_id — Kareem's live map depends on this
GET /api/v1/restaurants/{id}/menu	Hanaa → Kareem	items array with menu_item_id, price, variants — Kareem uses this in order creation
POST /api/v1/notifications	Sarhan → Basmala	type, recipient_id, message, order_id — Basmala's notification jobs consume this

4. Git workflow — non-negotiable rules (ignoreeeee)

4.1 Branch naming

```
feature/kareem-orders-payments
feature/sarhan-dispatch-realtime
feature/hanaa-restaurant-dashboard
feature/basmala-rider-testing

// For small fixes on your branch:
fix/kareem-state-machine-bug
fix/sarhan-pusher-connection
```

4.2 The two protected branches

- main — production-ready code only. Nobody pushes here directly. Ever.
- develop — integration branch. Everyone merges their features here via PR.

If you push directly to main or develop without a PR, you are responsible for fixing any breakage it causes for the whole team.

4.3 Daily workflow — follow this every single day

Step	What to do
1	git checkout develop && git pull origin develop — get everyone's latest work
2	git checkout feature/your-branch && git merge develop — bring updates into your branch
3	Write your code, commit often with clear messages (see 4.4 below)

Step	What to do
4	git push origin feature/your-branch — push your progress so teammates can see it
5	When a feature is done, open a Pull Request into develop on GitHub
6	Tag the right reviewer (see 4.5), wait for approval before merging

4.4 Commit message format

Every commit message must follow this format so the history is readable:

```
type(scope): short description
```

```
Types: feat | fix | test | docs | refactor | chore
```

```
Scope: orders | dispatch | restaurant | rider | payments | surge | auth
```

```
// Examples:
```

```
feat(orders): implement order state machine with transition guards
```

```
feat(surge): add multiplier strategy pattern
```

```
fix(dispatch): correct nearest-rider distance calculation
```

```
test(orders): add feature tests for invalid state transitions
```

```
docs(api): add Postman collection for order endpoints
```

```
refactor(auth): move token logic into AuthService
```

4.5 Code review ownership

PR touches...	Tag this reviewer
Auth, orders, payments, surge, DB schema	Kareem (architecture owner)
Dispatch, Redis queues, Pusher, GPS, security	Sarhan
Restaurant, menu, reviews, dashboards	Hanaa
Rider portal, notifications, tests, docs	Basmala
Any cross-module integration PR	Kareem + the module owner

5. Environment and secrets

5.1 The .env rule

The real .env file is never committed to GitHub. It is in .gitignore. The repo contains only .env.example with empty values:

```
// .env.example - commit this to the repo
APP_NAME=DeliverEats
DB_DATABASE=deliver_eats
DB_USERNAME=
DB_PASSWORD=

PUSHER_APP_ID=
PUSHER_APP_KEY=
PUSHER_APP_SECRET=

GOOGLE_MAPS_KEY=

STRIPE_KEY=
STRIPE_SECRET=

REDIS_HOST=127.0.0.1
REDIS_PORT=6379

// When you clone the repo, run:
cp .env.example .env
// Then fill in your own keys
```

Never share real API keys in WhatsApp, Discord, or any chat. Never paste them in GitHub issues or PR comments. If a key is accidentally committed, rotate it immediately.

6. Database rules

6.1 Migration ownership

Only create migrations for tables in your own module. If you need a column added to someone else's table, ask them to create that migration.

Member	Tables they own / migrate
Kareem	users, orders, order_items, order_history, payouts, surge_logs
Sarhan	rider_locations, dispatch_queue, push_events
Hanaa	restaurants, menu_categories, menu_items, item_variants, reviews
Basmala	notifications, rider_earnings, rider_deliveries

6.2 Foreign key naming

```
// Always: referenced_table_singular + _id
restaurant_id    // references restaurants.id
rider_id         // references users.id (riders are users with role=rider)
customer_id      // references users.id
order_id         // references orders.id
menu_item_id     // references menu_items.id

// WRONG
restaurantID     restaurant_ID     fk_restaurant
```

6.3 Every table must have these columns

```
$table->id();           // auto-increment primary key
$table->timestamps();   // created_at and updated_at – always
$table->softDeletes();  // deleted_at – for tables that should never hard-
delete
```

7. Folder structure

Every member follows this Laravel structure inside their module:

```
app/
  Http/
    Controllers/
      Api/
        OrderController.php    // Kareem
        DispatchController.php // Sarhan
        RestaurantController.php // Hanaa
        RiderController.php    // Basmala
    Services/
      OrderService.php
      DispatchService.php
      SurgeService.php
      PaymentService.php
    Jobs/
      AssignRiderJob.php
      RecalculateSurgeJob.php
      SendNotificationJob.php
    Events/
      OrderPlaced.php
      RiderAssigned.php
      RiderLocationUpdated.php
    Models/
      Order.php  Restaurant.php  Rider.php  MenuItem.php
  routes/
    api.php      // all API routes – one file, organized by module section
  tests/
    Feature/
```

```
OrderStateMachineTest.php
DispatchAlgorithmTest.php
SurgePricingTest.php
PaymentSplitTest.php
```

8. Routes file structure

All routes go in routes/api.php. Organize them in clearly labeled sections so nobody overwrites anyone else:

```
// =====
// AUTH ROUTES - Owner: Kareem
// =====
Route::post('/v1/register', [AuthController::class, 'register']);
Route::post('/v1/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {

    // =====
    // ORDER ROUTES - Owner: Kareem
    // =====
    Route::post('/v1/orders', [OrderController::class, 'store']);
    Route::get('/v1/orders/{id}', [OrderController::class, 'show']);
    Route::patch('/v1/orders/{id}/status', [OrderController::class,
'updateStatus']);

    // =====
    // DISPATCH & RIDER ROUTES - Owner: Sarhan
    // =====
    Route::put('/v1/rider-location', [DispatchController::class,
'updateLocation']);
    Route::get('/v1/riders/available', [DispatchController::class, 'available']);

    // =====
    // RESTAURANT ROUTES - Owner: Hanaa
    // =====
    Route::apiResource('/v1/restaurants', RestaurantController::class);
    Route::apiResource('/v1/menu-items', MenuItemController::class);

    // =====
    // RIDER PORTAL ROUTES - Owner: Basmala
    // =====
    Route::get('/v1/rider/deliveries', [RiderController::class, 'deliveries']);
    Route::get('/v1/rider/earnings', [RiderController::class, 'earnings']);

});
```

9. Communication rules

9.1 When to message the group

- Before starting work that might touch someone else's module
- When you finish a feature and open a PR — post the link in the group
- When you find a bug in someone else's code — tag them directly, don't fix it silently
- When you change any shared contract (API response shape, DB column name, event payload)

9.2 When NOT to push to develop

- Your code doesn't run (php artisan serve crashes)
- Any existing test is failing
- You haven't tested your own endpoints with Postman
- You have debug `dd()` or `var_dump()` left in the code

9.3 Weekly sync checklist

At the end of each week, every member answers these three questions in the group chat:

- What did I finish this week?
- What am I blocked on?
- What do I need from someone else next week?

This takes 2 minutes per person and prevents the situation where someone is stuck for 3 days without telling anyone.

10. Quick reference card

Print or pin this. When you're unsure, check here first.

Question	Answer
What case for DB columns?	snake_case — always
What case for PHP methods?	camelCase — always
What case for PHP classes?	PascalCase — always
What case for API routes?	kebab-case under <code>/api/v1/</code>
What case for JSON keys?	snake_case — always
Can I push to main?	No. Open a PR into develop only.

Question	Answer
Can I touch someone's module?	Only after talking to them + PR review
Where do secrets go?	.env only — never committed to Git
How do I name a migration?	timestamp_create_tablename_table
Who reviews arch/payment PRs?	Kareem
Who reviews real-time/dispatch PRs?	Sarhan
Who reviews restaurant PRs?	Hanaa
Who reviews rider/test PRs?	Basmala

“A project with clear rules ships faster than a project with smart people.” Kareem Said

Follow this document and the team will not have merge conflicts, naming fights, or broken builds.